ECOWATCH PROJECT

42-DAY INCEPTION REPORT

Environmental Monitoring and Issue Management Web Application

Project Name: EcoWatch
Project Duration: 42 Days
Project Type: Web Application Development
Team Size: 2 Members

Frontend Technologies: React.js, JavaScript, HTML, CSS, Vite
Backend Technologies: Node.js, Express.js
Database: MongoDB

Team Leader : Reet Gupta
Team Member : Shivani Sharma
Project Start Date: 1st September 2026
Project End Date: 15 October 2026
Institution/Organization: Eternal University

---

1. Introduction

EcoWatch is a web-based environmental monitoring application designed to help communities report, track, and manage environmental issues such as waste dumping, water pollution, water wastage, air pollution, and deforestation.

The application provides a structured workflow in which citizens can submit environmental reports, supervisors can review and assign issues, workers can update work progress, and administrators can monitor overall activities.

This 42-day inception report presents the project objectives, development plan, team responsibilities, technology stack, day-wise activities, expected outcomes, and future improvements.

2. Objectives of the Project

1. To develop a user-friendly environmental monitoring web application.
2. To enable citizens to report environmental issues.
3. To record issue descriptions, photographs, and location details.
4. To provide supervisors with issue review and assignment facilities.
5. To allow workers to update progress and provide completion evidence.
6. To provide administrators with dashboards and issue management features.
7. To establish an organized issue-tracking workflow.
8. To integrate the frontend, backend, and database.
9. To test application functionality and resolve identified errors.
10. To prepare complete project documentation.

3. Project Scope

The scope of EcoWatch includes:

- User registration and login interfaces.
- Citizen dashboard.
- Environmental issue reporting.
- Issue details and status tracking.
- Supervisor issue review and assignment.
- Worker task management.
- Work progress updates.
- Completion evidence and photographs.
- Admin dashboard and analytics.
- Backend API development.
- MongoDB database connectivity.
- Application testing and documentation.

4. Team Roles and Responsibilities

Team Member 1 – Frontend Development

Proposed responsibilities:

- Requirement analysis and UI planning.
- React project setup and folder organization.
- Development of reusable React components.
- Navbar, Sidebar, Footer, and dashboard development.
- Home, Login, and Register page development.
- Citizen, Admin, Supervisor, and Worker interfaces.
- Styling, responsive layout, and frontend testing.

- Frontend documentation.

Team Member 2 – Backend Development

Proposed responsibilities:

- Backend architecture and server setup.
- Node.js and Express.js configuration.
- MongoDB connection and database planning.
- API routes and controller development.
- Data validation and error handling.
- Authentication and authorization, where implemented.
- Frontend-backend integration.
- API testing, debugging, and backend documentation.

Both members collaborate on requirement analysis, workflow planning, integration, testing, debugging, and final documentation. The responsibilities should be updated to reflect the actual contributions of each member.

5. Day-Wise Project Development Report

Phase 1: Project Initiation and Planning (Days 1–5)

Day 1 – Project Introduction

The EcoWatch project concept was introduced, and the team discussed how a web application could help communities report environmental problems.

Day 2 – Problem Identification

The team identified key environmental problems, including waste dumping, water pollution, water wastage, air pollution, and deforestation.

Day 3 – Requirement Gathering

Initial functional requirements were identified, including issue reporting, user roles, status tracking, and task assignment.

Day 4 – Project Objectives and Scope

The project objectives, expected outcomes, and major features were outlined.

Day 5 – Technology Selection

The technologies required for development were selected, including React.js, JavaScript, CSS, Node.js, Express.js, and MongoDB.

Phase Outcome: Initial requirements, objectives, scope, and technology choices were documented.

Phase 2: System Design and Architecture (Days 6–10)

Day 6 – User Role Identification

The four primary roles—Admin, Citizen, Supervisor, and Worker—were identified.

Day 7 – Workflow Planning

The issue management workflow was planned: Report Submitted → Reviewed → Assigned → Work Started → Completed → Verified.

Day 8 – UI and Navigation Planning

The team planned the application screens, navigation structure, forms, dashboards, and reusable components.

Day 9 – Frontend Architecture

The React project structure was organized into components, layouts, pages, and assets.

Day 10 – Backend and Database Planning

The backend architecture, API responsibilities, database requirements, and frontend-backend communication were planned.

Phase Outcome: The preliminary application architecture and system workflow were established.

Phase 3: Frontend Development (Days 11–17)

Day 11 – React Project Setup

The React application was configured using Vite, and the initial project files were reviewed.

Day 12 – Navigation Components

The navigation and basic page layout were developed.

Day 13 – Reusable UI Components

Reusable components such as Sidebar, Navbar, StatCard, IssueCard, StatusBadge, and Footer were developed or organized.

Day 14 – Home Page Development

The home page layout and introductory content were prepared.

Day 15 – Login and Registration Pages

The login and registration interfaces were developed.

Day 16 – Citizen Dashboard

The citizen dashboard was developed to display issue-related information.

Day 17 – Issue Reporting Interface

The issue reporting form was prepared with fields for issue type, description, location, and photographs.

Phase Outcome: The main frontend foundation and citizen-facing screens were prepared.

Phase 4: Role-Based Dashboards and Features (Days 18–23)

Day 18 – Admin Dashboard

The administrator dashboard was developed to display an overview of environmental reports and application activity.

Day 19 – Admin Management Features

The interface for reviewing and managing reported issues was developed.

Day 20 – Analytics Interface

The analytics page was prepared to display issue-related summaries and statistics.

Day 21 – Citizen Issue Tracking

The screens for viewing submitted reports, issue details, and status information were developed.

Day 22 – Supervisor Dashboard

The supervisor interface was prepared for reviewing reports and assigning work.

Day 23 – Worker Dashboard

The worker interface was prepared for viewing assigned tasks and updating progress.

Phase Outcome: Role-based dashboards and major application screens were developed or organized.

Phase 5: Backend Development (Days 24–29)

Day 24 – Backend Setup

The backend project environment was configured using Node.js and Express.js.

Day 25 – Server Configuration

The server entry point, environment variables, and application configuration were prepared.

Day 26 – MongoDB Configuration

The database connection settings and MongoDB integration were configured or investigated.

Day 27 – Data Model Planning

Data structures for users, environmental issues, assignments, and issue statuses were planned.

Day 28 – API Route Development

Backend routes for relevant application operations were planned or implemented.

Day 29 – Controllers and Error Handling

Controller logic, request validation, and server error handling were developed or reviewed.

Phase Outcome: The backend foundation, database configuration, and initial API structure were prepared.

Phase 6: Integration and Workflow Development (Days 30–34)

Day 30 – Frontend-Backend Integration

The team began connecting the React frontend with backend API endpoints.

Day 31 – Issue Submission Integration

The issue reporting process was connected to or tested against the relevant backend functionality.

Day 32 – Issue Assignment Workflow

The supervisor-to-worker task assignment workflow was implemented or reviewed.

Day 33 – Status Updates

The process for updating issue progress and completion status was developed or tested.

Day 34 – Database and Integration Review

The team reviewed data persistence, API responses, and communication between the frontend, backend, and database.

Phase Outcome: The primary application workflows were connected or prepared for integration testing.

Phase 7: Testing and Debugging (Days 35–39)

Day 35 – Frontend Testing

The team checked page navigation, forms, dashboard layouts, and reusable components.

Day 36 – Backend API Testing

Available API endpoints were tested for expected responses and error handling.

Day 37 – Database Testing

Database connectivity and relevant data operations were checked.

Day 38 – Bug Fixing

Identified frontend, backend, configuration, and integration errors were investigated and corrected where possible.

Day 39 – Workflow Testing

The complete issue lifecycle was reviewed, from report submission through review, assignment, work completion, and verification.

Phase Outcome: Testing findings were reviewed, and identified issues were addressed where possible.

Phase 8: Final Review and Documentation (Days 40–42)

Day 40 – User Interface Review

The team reviewed the application's layout, navigation, consistency, and usability.

Day 41 – Project Documentation

The project overview, objectives, folder structure, technologies, features, and workflows were documented.

Day 42 – Final Project Review

The team reviewed the available application features, documented limitations, summarized the development process, and identified future improvements.

Phase Outcome: The project documentation and final review were prepared according to the implementation status.

6. Technology Stack

Technology	Purpose
React.js	Frontend development
JavaScript	Application logic
HTML	Page structure
CSS	Styling and layout
Vite	Frontend development and build tooling
Node.js	Backend runtime
Express.js	Backend API framework
MongoDB	Database storage
Git/GitHub	Version control, if configured

7. Expected Outcomes

The expected outcomes of the EcoWatch project are:

1. A digital platform for environmental issue reporting.
2. Role-based interfaces for citizens, administrators, supervisors, and workers.
3. A structured issue assignment and tracking workflow.
4. Backend APIs for application operations.
5. Database support for storing relevant information.
6. Improved visibility into the status of environmental reports.
7. Project documentation for future maintenance and development.

These outcomes should be marked as completed only after verifying the actual application.

8. Challenges and Risk Areas

Potential development challenges include:

- Organizing a maintainable React project structure.
- Managing navigation across multiple user roles.
- Configuring the backend and database connection.
- Validating user input and handling API errors.
- Integrating frontend components with backend services.
- Managing issue photographs and location information.
- Testing the complete issue management workflow.
- Coordinating work between two team members.

9. Future Improvements

Possible future improvements include:

- Map-based visualization of environmental issues.
- Integration of image upload and storage.
- Location services for accurate issue reporting.
- Notifications for status changes.
- Advanced analytics and reporting.
- Improved authentication and role-based access control.
- Mobile-responsive and accessibility improvements.

10. Conclusion

The EcoWatch project aims to provide a digital solution for reporting, assigning, monitoring, and managing environmental issues. The 42-day development plan covers project initiation, requirement gathering, system design, frontend development, backend development, integration, testing, and documentation.

The project adopts a role-based approach to help citizens, supervisors, workers, and administrators coordinate environmental issue resolution.

This report serves as a structured record of the proposed development activities. The final submitted version should include the actual dates, completed tasks, challenges, and individual contributions of both team members.

Approval and Sign-Off

Team Member 1: Reet Gupta

Team Member 2: Shivani Sharma

Project Guide/Mentor: Bhavna Sharma

Date: 28/8/2026

ECOWATCH PROJECT

42-DAY INCEPTION REPORT

Environmental Monitoring and Issue Management Web Application

Project Name: EcoWatch
Project Duration: 42 Days
Project Type: Web Application Development
Team Size: 2 Members

Frontend Technologies: React.js, JavaScript, HTML, CSS, Vite
Backend Technologies: Node.js, Express.js
Database: MongoDB

Team Leader: Reet Gupta
Team Member 2: Shivani Sharma
Project Start Date: 1/9/2026

Project End Date: 15/10/2026
Institution/Organization: Eternal Univesity

---

1. Introduction

EcoWatch is a web-based environmental monitoring application designed to help communities report, track, and manage environmental issues such as waste dumping, water pollution, water wastage, air pollution, and deforestation.

The application provides a structured workflow in which citizens can submit environmental reports, supervisors can review and assign issues, workers can update work progress, and administrators can monitor overall activities.

This 42-day inception report presents the project objectives, development plan, team responsibilities, technology stack, day-wise activities, expected outcomes, and future improvements.

2. Objectives of the Project

1. To develop a user-friendly environmental monitoring web application.
2. To enable citizens to report environmental issues.
3. To record issue descriptions, photographs, and location details.
4. To provide supervisors with issue review and assignment facilities.
5. To allow workers to update progress and provide completion evidence.
6. To provide administrators with dashboards and issue management features.
7. To establish an organized issue-tracking workflow.
8. To integrate the frontend, backend, and database.
9. To test application functionality and resolve identified errors.
10. To prepare complete project documentation.

3. Project Scope

The scope of EcoWatch includes:

- User registration and login interfaces.
- Citizen dashboard.
- Environmental issue reporting.
- Issue details and status tracking.

- Supervisor issue review and assignment.
- Worker task management.
- Work progress updates.
- Completion evidence and photographs.
- Admin dashboard and analytics.
- Backend API development.
- MongoDB database connectivity.
- Application testing and documentation.

4. Team Roles and Responsibilities

Team Member 1 – Frontend Development

Proposed responsibilities:

- Requirement analysis and UI planning.
- React project setup and folder organization.
- Development of reusable React components.
- Navbar, Sidebar, Footer, and dashboard development.
- Home, Login, and Register page development.
- Citizen, Admin, Supervisor, and Worker interfaces.
- Styling, responsive layout, and frontend testing.
- Frontend documentation.

Team Member 2 – Backend Development

Proposed responsibilities:

- Backend architecture and server setup.
- Node.js and Express.js configuration.
- MongoDB connection and database planning.
- API routes and controller development.
- Data validation and error handling.
- Authentication and authorization, where implemented.
- Frontend-backend integration.
- API testing, debugging, and backend documentation.

Both members collaborate on requirement analysis, workflow planning, integration, testing, debugging, and final documentation. The responsibilities should be updated to reflect the actual contributions of each member.

5. Day-Wise Project Development Report

Phase 1: Project Initiation and Planning (Days 1–5)

Day 1 – Project Introduction

The EcoWatch project concept was introduced, and the team discussed how a web application could help communities report environmental problems.

Day 2 – Problem Identification

The team identified key environmental problems, including waste dumping, water pollution, water wastage, air pollution, and deforestation.

Day 3 – Requirement Gathering

Initial functional requirements were identified, including issue reporting, user roles, status tracking, and task assignment.

Day 4 – Project Objectives and Scope

The project objectives, expected outcomes, and major features were outlined.

Day 5 – Technology Selection

The technologies required for development were selected, including React.js, JavaScript, CSS, Node.js, Express.js, and MongoDB.

Phase Outcome: Initial requirements, objectives, scope, and technology choices were documented.

Phase 2: System Design and Architecture (Days 6–10)

Day 6 – User Role Identification

The four primary roles—Admin, Citizen, Supervisor, and Worker—were identified.

Day 7 – Workflow Planning

The issue management workflow was planned: Report Submitted → Reviewed → Assigned → Work Started → Completed → Verified.

Day 8 – UI and Navigation Planning

The team planned the application screens, navigation structure, forms, dashboards, and reusable components.

Day 9 – Frontend Architecture

The React project structure was organized into components, layouts, pages, and assets.

Day 10 – Backend and Database Planning

The backend architecture, API responsibilities, database requirements, and frontend-backend communication were planned.

Phase Outcome: The preliminary application architecture and system workflow were established.

Phase 3: Frontend Development (Days 11–17)

Day 11 – React Project Setup

The React application was configured using Vite, and the initial project files were reviewed.

Day 12 – Navigation Components

The navigation and basic page layout were developed.

Day 13 – Reusable UI Components

Reusable components such as Sidebar, Navbar, StatCard, IssueCard, StatusBadge, and Footer were developed or organized.

Day 14 – Home Page Development

The home page layout and introductory content were prepared.

Day 15 – Login and Registration Pages

The login and registration interfaces were developed.

Day 16 – Citizen Dashboard

The citizen dashboard was developed to display issue-related information.

Day 17 – Issue Reporting Interface

The issue reporting form was prepared with fields for issue type, description, location, and photographs.

Phase Outcome: The main frontend foundation and citizen-facing screens were prepared.

Phase 4: Role-Based Dashboards and Features (Days 18–23)

Day 18 – Admin Dashboard

The administrator dashboard was developed to display an overview of environmental reports and application activity.

Day 19 – Admin Management Features

The interface for reviewing and managing reported issues was developed.

Day 20 – Analytics Interface

The analytics page was prepared to display issue-related summaries and statistics.

Day 21 – Citizen Issue Tracking

The screens for viewing submitted reports, issue details, and status information were developed.

Day 22 – Supervisor Dashboard

The supervisor interface was prepared for reviewing reports and assigning work.

Day 23 – Worker Dashboard

The worker interface was prepared for viewing assigned tasks and updating progress.

Phase Outcome: Role-based dashboards and major application screens were developed or organized.

Phase 5: Backend Development (Days 24–29)

Day 24 – Backend Setup

The backend project environment was configured using Node.js and Express.js.

Day 25 – Server Configuration

The server entry point, environment variables, and application configuration were prepared.

Day 26 – MongoDB Configuration

The database connection settings and MongoDB integration were configured or investigated.

Day 27 – Data Model Planning

Data structures for users, environmental issues, assignments, and issue statuses were planned.

Day 28 – API Route Development

Backend routes for relevant application operations were planned or implemented.

Day 29 – Controllers and Error Handling

Controller logic, request validation, and server error handling were developed or reviewed.

Phase Outcome: The backend foundation, database configuration, and initial API structure were prepared.

Phase 6: Integration and Workflow Development (Days 30–34)

Day 30 – Frontend-Backend Integration

The team began connecting the React frontend with backend API endpoints.

Day 31 – Issue Submission Integration

The issue reporting process was connected to or tested against the relevant backend functionality.

Day 32 – Issue Assignment Workflow

The supervisor-to-worker task assignment workflow was implemented or reviewed.

Day 33 – Status Updates

The process for updating issue progress and completion status was developed or tested.

Day 34 – Database and Integration Review

The team reviewed data persistence, API responses, and communication between the frontend, backend, and database.

Phase Outcome: The primary application workflows were connected or prepared for integration testing.

Phase 7: Testing and Debugging (Days 35–39)

Day 35 – Frontend Testing

The team checked page navigation, forms, dashboard layouts, and reusable components.

Day 36 – Backend API Testing

Available API endpoints were tested for expected responses and error handling.

Day 37 – Database Testing

Database connectivity and relevant data operations were checked.

Day 38 – Bug Fixing

Identified frontend, backend, configuration, and integration errors were investigated and corrected where possible.

Day 39 – Workflow Testing

The complete issue lifecycle was reviewed, from report submission through review, assignment, work completion, and verification.

Phase Outcome: Testing findings were reviewed, and identified issues were addressed where possible.

Phase 8: Final Review and Documentation (Days 40–42)

Day 40 – User Interface Review

The team reviewed the application's layout, navigation, consistency, and usability.

Day 41 – Project Documentation

The project overview, objectives, folder structure, technologies, features, and workflows were documented.

Day 42 – Final Project Review

The team reviewed the available application features, documented limitations, summarized the development process, and identified future improvements.

Phase Outcome: The project documentation and final review were prepared according to the implementation status.

6. Technology Stack

Technology	Purpose
React.js	Frontend development
JavaScript	Application logic
HTML	Page structure
CSS	Styling and layout
Vite	Frontend development and build tooling
Node.js	Backend runtime
Express.js	Backend API framework
MongoDB	Database storage
Git/GitHub	Version control, if configured

7. Expected Outcomes

The expected outcomes of the EcoWatch project are:

1. A digital platform for environmental issue reporting.
2. Role-based interfaces for citizens, administrators, supervisors, and workers.
3. A structured issue assignment and tracking workflow.
4. Backend APIs for application operations.
5. Database support for storing relevant information.
6. Improved visibility into the status of environmental reports.
7. Project documentation for future maintenance and development.

These outcomes should be marked as completed only after verifying the actual application.

8. Challenges and Risk Areas

Potential development challenges include:

- Organizing a maintainable React project structure.
- Managing navigation across multiple user roles.

- Configuring the backend and database connection.
- Validating user input and handling API errors.
- Integrating frontend components with backend services.
- Managing issue photographs and location information.
- Testing the complete issue management workflow.
- Coordinating work between two team members.

9. Future Improvements

Possible future improvements include:

- Map-based visualization of environmental issues.
- Integration of image upload and storage.
- Location services for accurate issue reporting.
- Notifications for status changes.
- Advanced analytics and reporting.
- Improved authentication and role-based access control.
- Mobile-responsive and accessibility improvements.

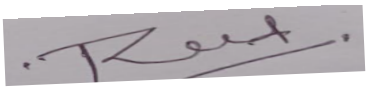
10. Conclusion

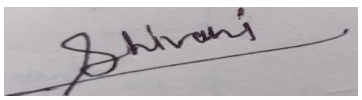
The EcoWatch project aims to provide a digital solution for reporting, assigning, monitoring, and managing environmental issues. The 42-day development plan covers project initiation, requirement gathering, system design, frontend development, backend development, integration, testing, and documentation.

The project adopts a role-based approach to help citizens, supervisors, workers, and administrators coordinate environmental issue resolution.

This report serves as a structured record of the proposed development activities. The final submitted version should include the actual dates, completed tasks, challenges, and individual contributions of both team members.

Approval and Sign-Off

Team Leader: 

Team Member: 

Project Guide/Mentor:

Date: 15 October 2026